National University of computer and emerging sciences

Artificial Intelligence Lab (AI-2002)

Semester: Spring 2026
Section: BCS-6A & BCS-6F
Course Instructor: Mr. Abdullah Shaikh

LAB # 04 -TASKS

Introduction to Uninformed Search Algorithms and their Implementation

Submission Instructions:

- Perform all tasks. Late submissions will not be considered.
- Submit a .ipynb file with the filename formatted as:
  Lab 04 - Your Name - Roll Number
- Upload the file on GCR.

TASK # 01

An AI-powered autonomous rescue robot is deployed inside a partially collapsed research facility after an earthquake.

The building layout is modeled as a graph, where:

- Each node represents a room or area
- Each edge represents a safe passage
- The robot starts from the Control Room
- The goal is to reach the Survivor Room
- Some paths may lead to dead ends
- The robot must systematically explore the environment

The robot's navigation system must implement uninformed search algorithms to explore the environment.

Graph Representation (Adjacency List)

Control Room → Storage A, Storage B
Storage A → Lab 1, Lab 2
Storage B → Server Room
Lab 1 → Debris 1
Lab 2 → Debris 2
Debris 1 → Maintenance Corridor
Debris 2 → Maintenance Corridor
Server Room → Charging Bay
Charging Bay → Maintenance Corridor
Maintenance Corridor → Survivor Room
Survivor Room → (No outgoing edges)

You are required to implement AI search algorithms to help the rescue robot locate the **Survivor Room** starting from the **Control Room**.

The robot must stop searching immediately when the goal is found.

Task 01 (a): BFS – Implement Breadth-First Search using a queue; display the order of node expansion and the path to the goal.

Task 02 (b): DFS Recursive – Implement Depth-First Search using recursion; display the order of node expansion and the path to the goal.

Task 03 (c): DFS Iterative – Implement Depth-First Search using an explicit stack (no recursion); display the order of node expansion and the path to the goal.

TASK # 02

A city has been hit by a massive earthquake. An AI-powered disaster response robot has been deployed to deliver medical supplies to victims.

The city map is represented as a graph, where:

- Nodes represent locations (hospitals, shelters, streets, blocked zones).
- Edges represent roads between locations, with costs representing travel time (minutes) or energy consumption.
- Some roads are longer, others shorter but more dangerous, so the robot must find the least-cost path, not just the shortest path in number of steps.

The robot starts at the Central Command Center and must reach the Hospital Zone where critically injured people are trapped.

Graph Layout (Nodes & Edge Costs)

From	To	Cost (minutes)
Central Command	Shelter A	4
Central Command	Shelter B	2
Shelter A	Street 1	5
Shelter A	Street 2	3
Shelter B	Street 2	2
Street 1	Hospital Zone	7
Street 2	Hospital Zone	4
Street 2	Street 3	6
Street 3	Hospital Zone	2

You are required to implement Uniform Cost Search (UCS) to help the robot find the least-cost path from Central Command to Hospital Zone.

- Implement UCS using a priority queue (or min-heap) where the priority is the cumulative path cost.
- Display: The order in which nodes are expanded and the final path from start to goal along with total cost

TASK # 03

You are designing an AI agent for a treasure hunt game.

- The agent starts at the Entrance Room and must reach one of the Treasure Rooms.
- Each room is connected to other rooms via doors, and each door has a movement cost.
- Some rooms are traps - passing through them increases cost but does not provide treasure.
- Each treasure room has a treasure value, and the agent's utility function is:

Utility = Treasure Value – Path Cost

The agent's goal is to maximize its utility while safely reaching a treasure room.

Graph Representation

The game world is represented as a graph with rooms as nodes and edges with movement costs:

```
graph = {
    "Entrance": [("Room A", 2), ("Room B", 3)],
    "Room A": [("Room C", 2), ("Room D", 5)],
    "Room B": [("Room D", 2)],
    "Room C": [("Treasure Room 1", 4)],
    "Room D": [("Treasure Room 2", 3), ("Trap Room", 1)],
    "Trap Room": [("Treasure Room 3", 2)],
}
```

```
treasure_values = {
    "Treasure Room 1": 10,
    "Treasure Room 2": 8,
    "Treasure Room 3": 15
}
```

- Design a utility-based agent to find the Treasure Room with maximum utility.
- Implement an appropriate uninformed search algorithm (you decide which is best based on the scenario).
- Display the following: Order of node expansion (rooms visited in order), Path taken to the selected treasure room, and Total utility (treasure value – total path cost)

Expected Output Format (Example)

```
Node expansion order: Entrance, Room B, Room D, Trap Room, Treasure Room 3
Path to goal: Entrance -> Room B -> Room D -> Trap Room -> Treasure Room 3
Total utility: 15 - 6 = 9
```

Explanation:

- Path cost = sum of edge costs along the path
- Utility = treasure value – path cost

THE END!